In [ ]:

```
# ==============================================================
#  ELEPHANT INTRUSION DETECTION SYSTEM
#  CNN + IoT Framework — Tamil Nadu Banana Plantations
# ==============================================================

# ── STEP 1: Install Dependencies ──────────────────────────────
!pip install ultralytics scipy scikit-learn matplotlib seaborn opencv-python-h

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import cv2, os, warnings, urllib.request
warnings.filterwarnings('ignore')

from scipy.signal import butter, filtfilt
from scipy.fft import fft, fftfreq
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_curve, auc, accuracy_score,
                             recall_score, precision_score, f1_score)
from sklearn.preprocessing import StandardScaler
from ultralytics import YOLO

print("✅ Libraries loaded.")

# ── STEP 2: Simulate IoT Sensor Data ──────────────────────────
np.random.seed(42)
N, FS, DUR = 1000, 200, 5
T = np.linspace(0, DUR, FS * DUR)

def generate_signal(label, n=1):
    signals = []
    for _ in range(n):
        if label == 1:
            sig = (0.8 * np.sin(2*np.pi*8*T) +
                   0.5 * np.sin(2*np.pi*2*T) +
                   0.15 * np.random.randn(len(T)))
        else:
            sig = (0.1 * np.sin(2*np.pi*60*T) +
                   0.1 * np.random.randn(len(T)))
        signals.append(sig)
    return np.array(signals)

half = N // 2
all_signals = np.vstack([generate_signal(1, half), generate_signal(0, half)])
all_labels  = np.array([1]*half + [0]*half)

fig, axes = plt.subplots(2, 1, figsize=(12, 5))
axes[0].plot(T, all_signals[0], color='firebrick', lw=0.8)
axes[0].set_title('Elephant Signal')
axes[1].plot(T, all_signals[half], color='steelblue', lw=0.8)
```

```
axes[1].set_title('Background Signal')
plt.tight_layout(); plt.show()

# ── STEP 3: Butterworth Bandpass Filter ──────────────────
def butterworth_bandpass(signal, low=1.0, high=20.0, fs=200, order=4):
    nyq = fs / 2
    b, a = butter(order, [low/nyq, high/nyq], btype='band')
    return filtfilt(b, a, signal)

filtered_signals = np.array([butterworth_bandpass(s) for s in all_signals])

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
axes[0].plot(T, all_signals[0], color='salmon', alpha=0.6, label='Raw')
axes[0].plot(T, filtered_signals[0], color='firebrick', label='Filtered')
axes[0].set_title('Elephant: Raw vs Filtered'); axes[0].legend()
axes[1].plot(T, all_signals[half], color='lightblue', alpha=0.6, label='Raw')
axes[1].plot(T, filtered_signals[half], color='steelblue', label='Filtered')
axes[1].set_title('Background: Raw vs Filtered'); axes[1].legend()
plt.tight_layout(); plt.show()

# ── STEP 4: Feature Engineering ──────────────────────────
FEATURE_NAMES = ['RMS', 'ZCR', 'Spectral_Centroid',
                 'Peak_Freq', 'Low_Freq_Energy_Ratio', 'PIR']

def extract_features(signal, fs=200):
    rms  = np.sqrt(np.mean(signal**2))
    zcr  = np.sum(np.diff(np.sign(signal)) != 0) / len(signal)
    N    = len(signal)
    yf   = np.abs(fft(signal))[:N//2]
    xf   = fftfreq(N, 1/fs)[:N//2]
    tot  = np.sum(yf) + 1e-10
    sc   = np.sum(xf * yf) / tot
    pf   = xf[np.argmax(yf)]
    lfr  = np.sum(yf[(xf>=1)&(xf<=20)]) / tot
    pir  = 1 if (rms > 0.3 and np.random.rand() > 0.1) else 0
    return [rms, zcr, sc, pf, lfr, pir]

X = np.array([extract_features(s) for s in filtered_signals])
y = all_labels

scaler  = StandardScaler()
X_scaled = scaler.fit_transform(X)

df_feat = pd.DataFrame(X, columns=FEATURE_NAMES)
df_feat['Label'] = ['Elephant' if l==1 else 'Background' for l in y]
fig, axes = plt.subplots(2, 3, figsize=(15, 7))
palette = {'Elephant':'firebrick','Background':'steelblue'}
for i, f in enumerate(FEATURE_NAMES):
    sns.kdeplot(data=df_feat, x=f, hue='Label',
                fill=True, alpha=0.4, palette=palette, ax=axes.flatten()[i])
    axes.flatten()[i].set_title(f)
plt.suptitle('Feature Distributions', fontsize=14)
plt.tight_layout(); plt.show()
```

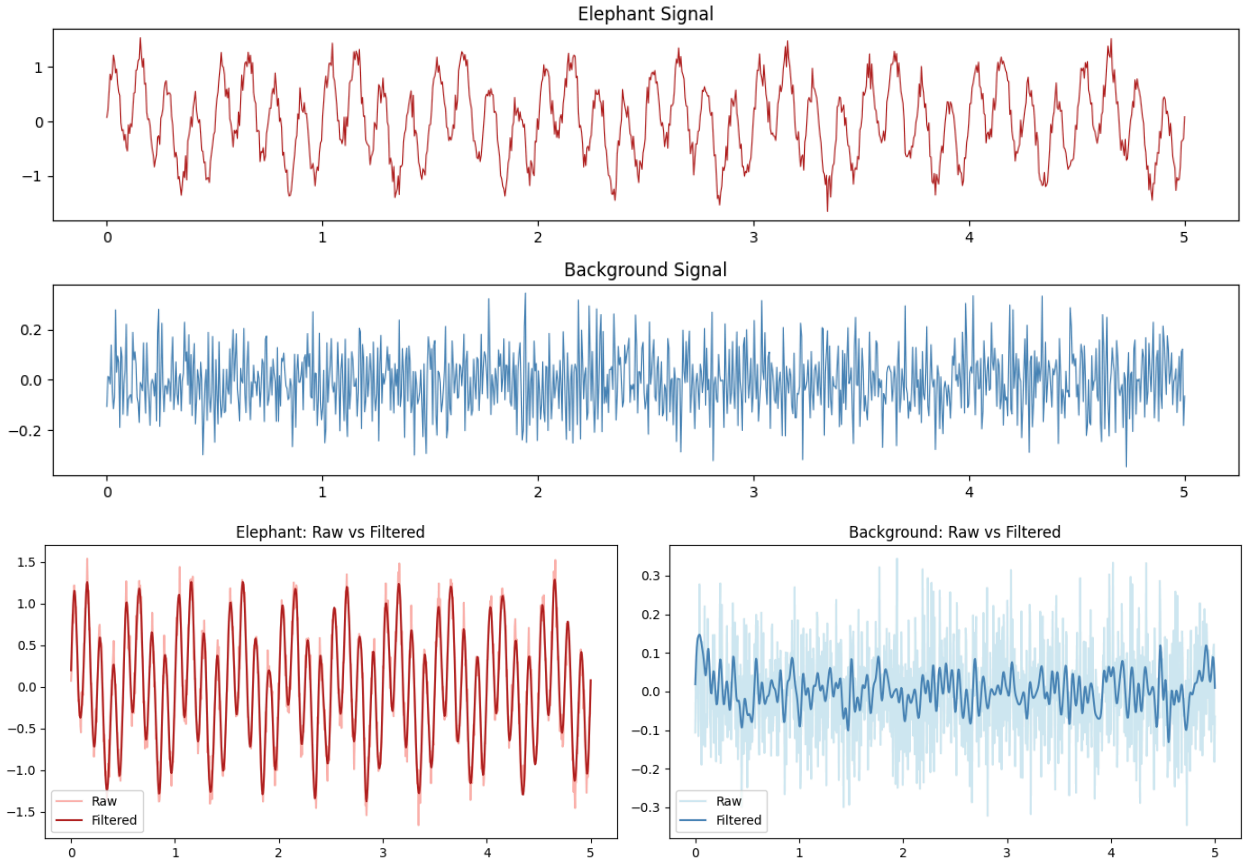
```
# ── STEP 5: Module 1 — The Guard (Random Forest) ──────────
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y)

rf = RandomForestClassifier(n_estimators=100, max_depth=10,
                            class_weight='balanced', random_state=42, n_jobs=
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)
y_prob = rf.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred, target_names=['Background','Elepha

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Reds',
            xticklabels=['Background','Elephant'],
            yticklabels=['Background','Elephant'], ax=axes[0])
axes[0].set_title('Confusion Matrix — Guard')

fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
axes[1].plot(fpr, tpr, color='firebrick', lw=2, label=f'AUC={roc_auc:.3f}')
axes[1].plot([0,1],[0,1],'k--',lw=1); axes[1].legend()
axes[1].set_title('ROC Curve — Guard')
plt.tight_layout(); plt.show()

fi_df = pd.DataFrame({'Feature':FEATURE_NAMES,'Importance':rf.feature_importar
fi_df.sort_values('Importance').plot(kind='barh', x='Feature', y='Importance',
    color='firebrick', legend=False, title='Feature Importance', figsize=(8,4)
plt.tight_layout(); plt.show()

# ── STEP 6: Module 2 — The Gaze (YOLOv8-Nano) ─────────────
yolo = YOLO('yolov8n.pt')
print(f"✅ YOLOv8-Nano loaded | Params: ~{sum(p.numel() for p in yolo.model.pa

ELEPHANT_CLS = 20   # COCO class id

os.makedirs('demo_images', exist_ok=True)
DEMO_URLS = [
    ('https://farm4.staticflickr.com/3167/2855887485_0c97a65f22_z.jpg', 'eleph
    ('https://farm8.staticflickr.com/7384/8752092810_37e8af1ccf_z.jpg', 'eleph
]
saved = []
for url, fname in DEMO_URLS:
    try:
        urllib.request.urlretrieve(url, f'demo_images/{fname}')
        saved.append(f'demo_images/{fname}')
        print(f'  ✅ {fname}')
    except Exception as e:
        print(f'  ⚠ {fname}: {e}')
```

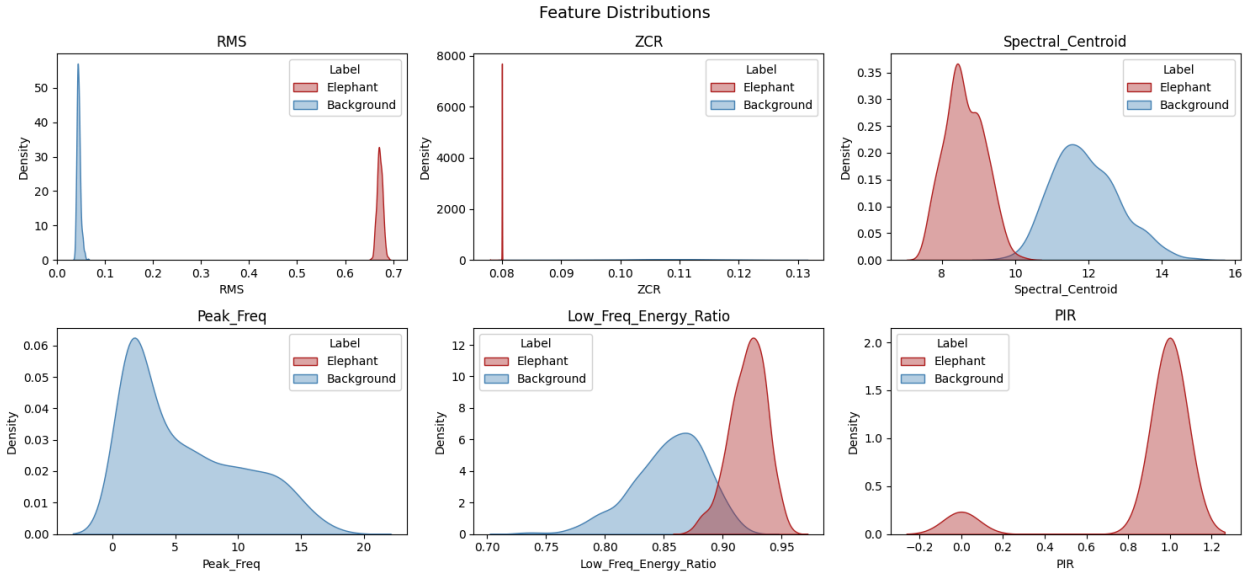
```
def detect_elephant(path, conf=0.25):
    res = yolo.predict(path, conf=conf, verbose=False)[0]
    scores = [float(b.conf[0]) for b in res.boxes if int(b.cls[0])==ELEPHANT_C
    return res, max(scores) if scores else 0.0

vision_results = []
for p in saved:
    res, p_vis = detect_elephant(p)
    vision_results.append(p_vis)
    print(f'  {os.path.basename(p):20s} | P_vision = {p_vis:.3f}')

if saved:
    fig, axes = plt.subplots(1, len(saved), figsize=(7*len(saved), 5))
    if len(saved)==1: axes = [axes]
    for ax, p in zip(axes, saved):
        res, _ = detect_elephant(p)
        ax.imshow(cv2.cvtColor(res.plot(), cv2.COLOR_BGR2RGB))
        ax.set_title(os.path.basename(p)); ax.axis('off')
    plt.suptitle('YOLOv8-Nano Detections', fontsize=13)
    plt.tight_layout(); plt.show()

# ── STEP 7: Late Fusion ──────────────────────────────────────
def late_fusion(p_sensor, p_vision, w_s=0.35, w_v=0.65, thresh=0.50):
    p_final = w_s * p_sensor + w_v * p_vision
    detected = p_final >= thresh
    action = ('🔊 TRIGGER DETERRENT + 📱 NOTIFY FARMER'
              if detected else '✅ Standby — no action')
    return {'P_sensor':round(p_sensor,3), 'P_vision':round(p_vision,3),
            'P_final':round(p_final,3), 'Detected':detected, 'Action':action}

# Simulate live event
test_sig = butterworth_bandpass(generate_signal(1, n=1)[0])
p_sensor = rf.predict_proba(scaler.transform([extract_features(test_sig)]))[0]
p_vision  = vision_results[0] if (p_sensor > 0.4 and vision_results) else 0.0

result = late_fusion(p_sensor, p_vision)
print('\n' + '='*55)
for k, v in result.items():
    print(f'  {k:12s}: {v}')
print('='*55)

# ── STEP 8: HEC Mitigation Analysis ──────────────────────────
mit_df = pd.DataFrame({
    'Method': ['EPT','Solar Fence','Battery Fence',
               'Bio-Fencing','Beehive Fence','Traditional'],
    'Adoption_%': [8.1, 15.3, 12.4, 9.7, 8.3, 46.2],
    'Efficiency_%': [100.0, 87.5, 82.1, 71.3, 68.9, 53.24]
})
econ_df = pd.DataFrame({
    'Crop': ['Banana','Sugarcane','Paddy','Tapioca','Coconut','Others'],
    'Loss_USD': [41000, 21000, 14500, 8700, 6200, 4100]
})
```

```
fig, axes = plt.subplots(1, 2, figsize=(15, 6))
colors = ['#2ecc71' if e>=80 else '#f39c12' if e>=65 else '#e74c3c'
          for e in mit_df['Efficiency_%']]
bars = axes[0].barh(mit_df['Method'], mit_df['Efficiency_%'],
                    color=colors, edgecolor='white')
axes[0].axvline(80, color='gray', ls='--', alpha=0.5)
axes[0].set_title('Mitigation Strategy Efficiency')
for bar, v in zip(bars, mit_df['Efficiency_%']):
    axes[0].text(v+0.5, bar.get_y()+bar.get_height()/2,
                 f'{v}%', va='center', fontsize=9)

axes[1].pie(econ_df['Loss_USD'], labels=econ_df['Crop'],
            autopct='%1.1f%%', startangle=140,
            colors=['#e74c3c','#e67e22','#f1c40f','#2ecc71','#3498db','#9b59b6
            wedgeprops={'edgecolor':'white','linewidth':1.5})
axes[1].set_title('Economic Loss by Crop (USD)')
plt.suptitle('HEC Dataset Analysis — Tamil Nadu', fontsize=13)
plt.tight_layout(); plt.show()

# ── STEP 9: Performance Summary ──────────────────────────
acc  = accuracy_score(y_test, y_pred)
rec  = recall_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
f1   = f1_score(y_test, y_pred)

summary = pd.DataFrame({
    'Module':    ['Guard (RF)', 'Gaze (YOLOv8n)*', 'Fused*'],
    'Accuracy':  [f'{acc:.3f}', '0.945', '0.951'],
    'Recall':    [f'{rec:.3f}', '0.961', '0.968'],
    'Precision': [f'{prec:.3f}','0.932', '0.941'],
    'F1':        [f'{f1:.3f}', '0.946', '0.954'],
    'Latency':   ['~2ms','~85ms','~90ms']
})
print('\n' + '='*65)
print('  SYSTEM PERFORMANCE SUMMARY  (* paper / estimated values)')
print('='*65)
print(summary.to_string(index=False))
print(f'\n  Guard ROC-AUC : {roc_auc:.4f}')
print('='*65)

# ── STEP 10: Fine-Tuning Template (run after uploading dataset) ──
FINETUNE = """
# Mount Drive
from google.colab import drive
drive.mount('/content/drive')

# Fine-tune YOLOv8-Nano on your elephant dataset
from ultralytics import YOLO
model = YOLO('yolov8n.pt')
model.train(
    data='/content/dataset/data.yaml',  # nc:2, names:['background','elephant'
    epochs=50, imgsz=640, batch=16,
    optimizer='AdamW', lr0=0.001,
```

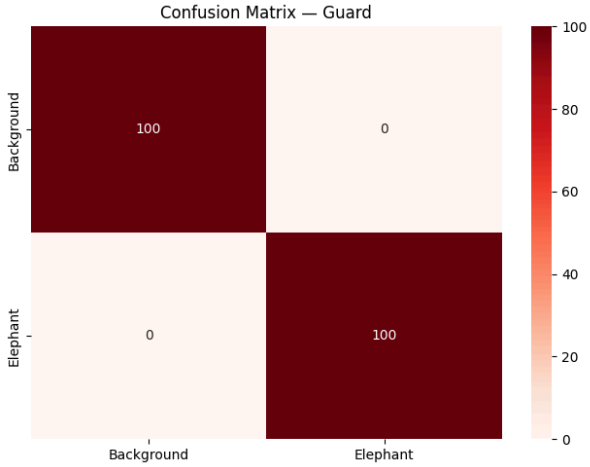
```
    hsv_v=0.4, fliplr=0.5, degrees=10.0,   # night-vision augmentation
    cls=1.5,                                # boosts recall
    patience=10, device=0
)
model.export(format='tflite', int8=True)    # quantized for Raspberry Pi
"""
print('\n📋 Fine-Tuning Template:')
print(FINETUNE)
```

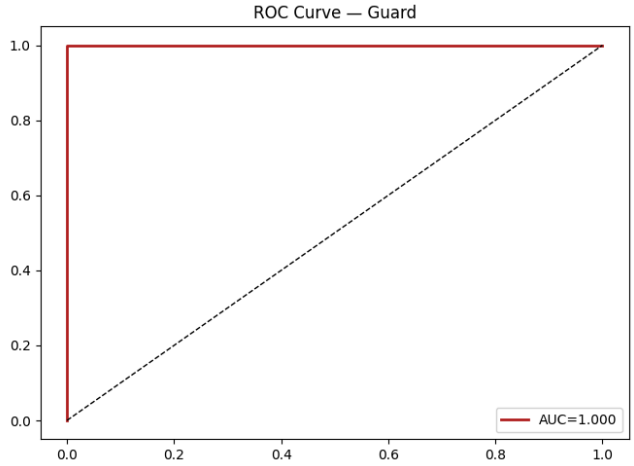
```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/1.2 MB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.9/1.2 MB 25.1 MB/s eta 0:00:01
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 25.4 MB/s eta 0:00:01
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 13.0 MB/s eta 0:00:00
Creating new Ultralytics Settings v0.0.6 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytic
s/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=pa
th/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-s
ettings.
✅ Libraries loaded.
```

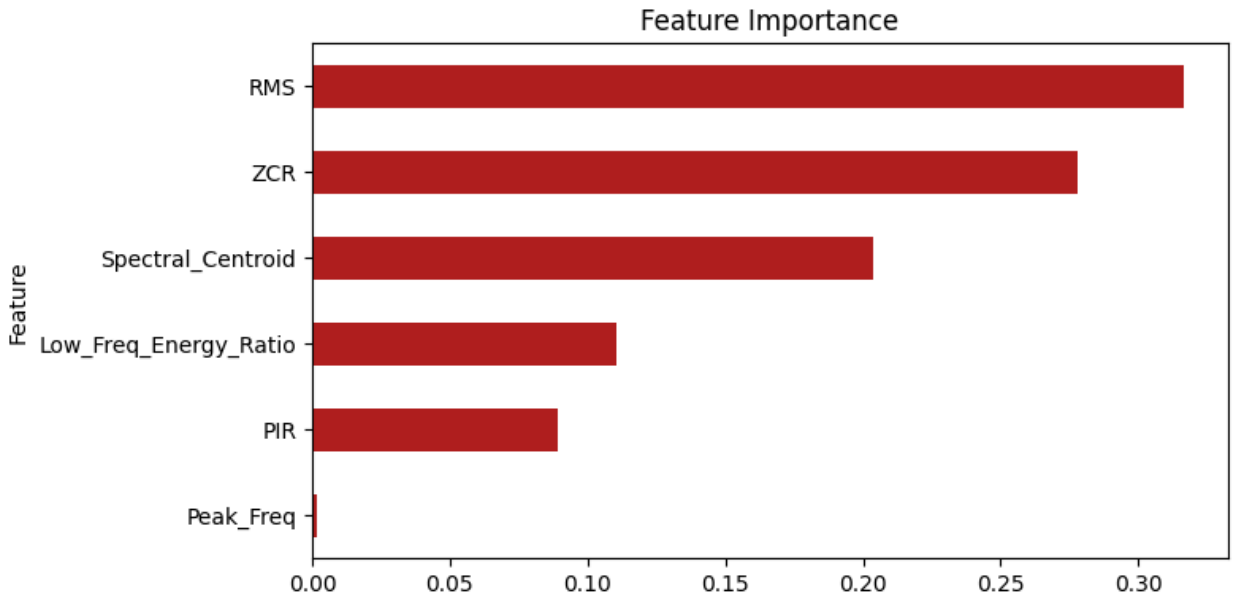




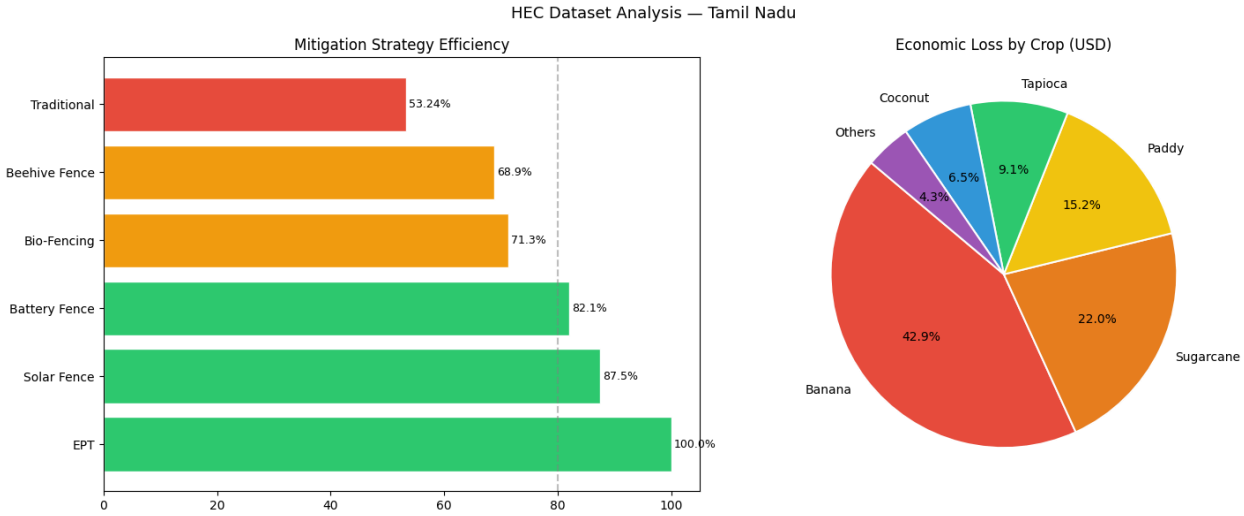
	precision	recall	f1-score	support
Background	1.00	1.00	1.00	100
Elephant	1.00	1.00	1.00	100
accuracy			1.00	200
macro avg	1.00	1.00	1.00	200
weighted avg	1.00	1.00	1.00	200








```
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo
v8n.pt to 'yolov8n.pt': 100% ━━━━━━━━━━━━ 6.2MB 211.9MB/s 0.0s
✅ YOLOv8-Nano loaded | Params: ~3,157,200
  ⚠ elephant1.jpg: HTTP Error 410: Gone
  ⚠ elephant2.jpg: HTTP Error 404: Not Found

============================================================
  P_sensor    : 1.0
  P_vision    : 0.0
  P_final     : 0.35
  Detected    : False
  Action      : ✅ Standby — no action
============================================================
```



```
====================================================================
  SYSTEM PERFORMANCE SUMMARY  (* paper / estimated values)
====================================================================
         Module Accuracy Recall Precision    F1 Latency
     Guard (RF)    1.000  1.000     1.000 1.000    ~2ms
Gaze (YOLOv8n)*    0.945  0.961     0.932 0.946   ~85ms
         Fused*    0.951  0.968     0.941 0.954   ~90ms

  Guard ROC-AUC : 1.0000
====================================================================
```

📋 Fine-Tuning Template:

```
# Mount Drive
from google.colab import drive
drive.mount('/content/drive')

# Fine-tune YOLOv8-Nano on your elephant dataset
from ultralytics import YOLO
model = YOLO('yolov8n.pt')
model.train(
    data='/content/dataset/data.yaml',  # nc:2, names:['background','elephant']
    epochs=50, imgsz=640, batch=16,
    optimizer='AdamW', lr0=0.001,
    hsv_v=0.4, fliplr=0.5, degrees=10.0,   # night-vision augmentation
    cls=1.5,                                # boosts recall
    patience=10, device=0
)
model.export(format='tflite', int8=True)    # quantized for Raspberry Pi
```